

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1**



Android Basics With Compose

Oleh:

Rika Fauliana Rahmi NIM. 2410817120017

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MARET 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basics With Compose Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rika Fauliana Rahmi
NIM : 2410817120017

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	8
B. Output Program	13
C. Pembahasan	16
D. Tautan Git	18

DAFTAR GAMBAR

Gambar 1 Tampilan Awal Aplikasi.....	6
Gambar 2 Tampilan Dadu Setelah Di-Roll	7
Gambar 3 Tampilan Roll Dadu Double.....	7
Gambar 4 Screenshot Hasil Jawaban Soal 1 XML.....	13
Gambar 5 Screenshot Hasil Jawaban Soal 1 XML.....	14
Gambar 6 Screenshot Hasil Jawaban Soal 1 XML.....	14
Gambar 7 Screenshot Hasil Jawaban Soal 1 Compose	15
Gambar 8 Screenshot Hasil Jawaban Soal 1 Compose	15
Gambar 9 Screenshot Hasil Jawaban Soal 1 Compose	16

DAFTAR TABEL

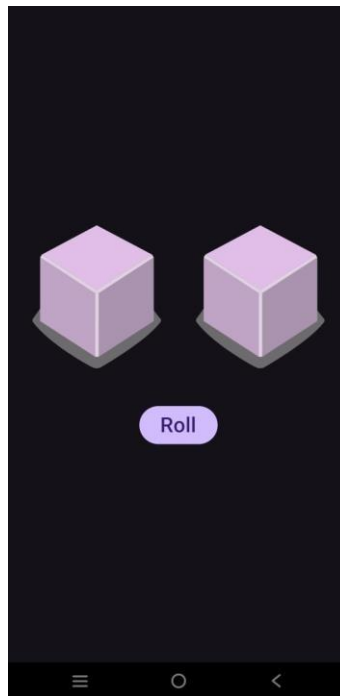
Tabel 1 Source Code MainActivity.kt XML Soal 1	8
Tabel 2 Source Code activity_main.xml Soal 1	9
Tabel 3 Source Code MainActivity.kt Compose Soal 1	10

SOAL 1

Soal Praktikum:

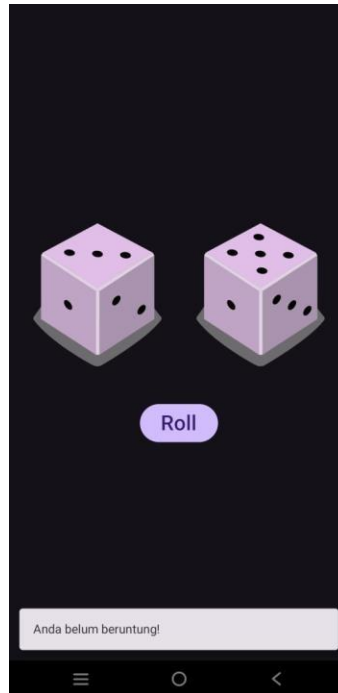
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol “Roll”. Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



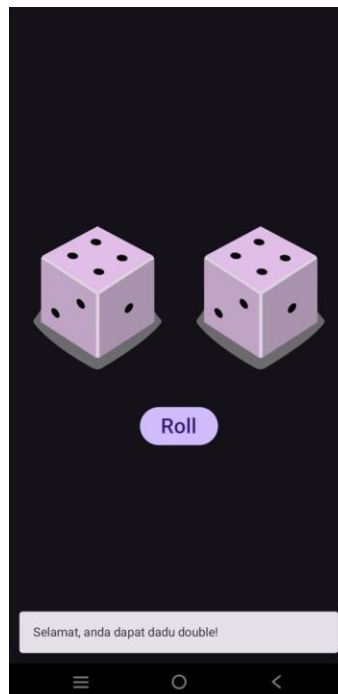
Gambar 1 Tampilan Awal Aplikasi

2. Setelah user menekan tombol “Roll” maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan “Anda belum beruntung!” seperti yang dapat dilihat pada Gambar 2.



Gambar 2 Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3 Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut:
https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp=sharing

Additional:

ESSAY

1. Riset dan *cite* **artikel/jurnal/buku** mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatlah opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan. **
2. Apa yang membedakan OOP pada Kotlin dengan OOP pada Java? *
3. Sebutkan, dan jelaskan mengenai SOLID Principles *
4. Apa perbedaan antara Class dan Data Class pada Kotlin ?

A. Source Code

MainActivity.kt XML

Tabel 1 Source Code MainActivity.kt XML Soal 1

1	package com.example.modul1xml
2	
3	import android.os.Bundle
4	import android.widget.Button
5	import android.widget.ImageView
6	import android.widget.Toast
7	import androidx.appcompat.app.AppCompatActivity
8	
9	class MainActivity : AppCompatActivity() {
10	
11	override fun onCreate(savedInstanceState: Bundle?) {
12	super.onCreate(savedInstanceState)
13	setContentView(R.layout.activity_main)
14	
15	val ivDice1: ImageView =
	findViewById(R.id.ivDice1)

16	val ivDice2: ImageView =
	findViewById(R.id.ivDice2)
17	val btnRoll: Button = findViewById(R.id.btnRoll)
18	
19	btnRoll.setOnClickListener {
20	val diceValue1 = (1..6).random()
21	val diceValue2 = (1..6).random()
22	
23	
	ivDice1.setImageResource(getDiceImage(diceValue1))
24	
	ivDice2.setImageResource(getDiceImage(diceValue2))
25	
26	if (diceValue1 == diceValue2) {
27	Toast.makeText(this, "Selamat, anda
	dapat dadu double!", Toast.LENGTH_SHORT).show()
28	} else {
29	Toast.makeText(this, "Anda belum
	beruntung!", Toast.LENGTH_SHORT).show()
30	}
31	}
32	}
33	private fun getDiceImage(value: Int): Int {
34	return when (value) {
35	1 -> R.drawable.dice_1
36	2 -> R.drawable.dice_2
37	3 -> R.drawable.dice_3
38	4 -> R.drawable.dice_4
39	5 -> R.drawable.dice_5
40	6 -> R.drawable.dice_6
41	else -> R.drawable.dice_0
42	}
43	}
44	}

activity_main.xml

Tabel 2 Source Code activity_main.xml Soal 1

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:tools="http://schemas.android.com/tools"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:gravity="center"

8	android:background="#121212">
9	
10	<LinearLayout
11	android:layout_width="wrap_content"
12	android:layout_height="wrap_content"
13	android:orientation="horizontal"
14	android:layout_marginBottom="32dp">
15	
16	<ImageView
17	android:id="@+id/ivDice1"
18	android:layout_width="120dp"
19	android:layout_height="120dp"
20	android:layout_marginEnd="16dp"
21	android:src="@drawable/dice_0"
22	android:contentDescription="Dice 1" />
23	
24	<ImageView
25	android:id="@+id/ivDice2"
26	android:layout_width="120dp"
27	android:layout_height="120dp"
28	android:src="@drawable/dice_0"
29	android:contentDescription="Dice 2" />
30	</LinearLayout>
31	
32	<Button
33	android:id="@+id/btnRoll"
34	android:layout_width="wrap_content"
35	android:layout_height="wrap_content"
36	android:text="Roll"
37	android:textAllCaps="false"
38	tools:ignore="HardcodedText"
3	android:backgroundTint="#D1C4E9"/>
9	
4	</LinearLayout>
0	
4	
1	

MainActivity.kt Compose

Tabel 3 Source Code MainActivity.kt Compose Soal 1

1	package com.example.modullcompose
2	
3	import android.os.Bundle
4	import android.widget.Toast
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent

```

7 import androidx.compose.foundation.Image
8 import androidx.compose.foundation.background
9 import androidx.compose.foundation.layout.Arrangement
10 import androidx.compose.foundation.layout.Column
11 import androidx.compose.foundation.layout.Row
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.material3.Button
16 import androidx.compose.material3.ButtonDefaults
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.Composable
19 import androidx.compose.runtime.getValue
20 import androidx.compose.runtime.mutableIntStateOf
21 import androidx.compose.runtime.remember
22 import androidx.compose.runtime.setValue
23 import androidx.compose.ui.Alignment
24 import androidx.compose.ui.Modifier
25 import androidx.compose.ui.graphics.Color
26 import androidx.compose.ui.platform.LocalContext
27 import androidx.compose.ui.res.painterResource
28 import androidx.compose.ui.unit.dp
29 import androidx.compose.ui.unit.sp
30
31 class MainActivity : ComponentActivity() {
32     override fun onCreate(savedInstanceState: Bundle?) {
33         super.onCreate(savedInstanceState)
34         setContent {
35             DiceRollerApp()
36         }
37     }
38 }
39
40 @Composable
41 fun DiceRollerApp() {
42     var diceValue1 by remember { mutableIntStateOf(0) }
43     var diceValue2 by remember { mutableIntStateOf(0) }
44
45     val context = LocalContext.current
46
47     Column(
48         modifier = Modifier
49             .fillMaxSize()
50             .background(Color(0xFF121212)),
51         horizontalAlignment = Alignment.CenterHorizontally,
52         verticalArrangement = Arrangement.Center

```

```

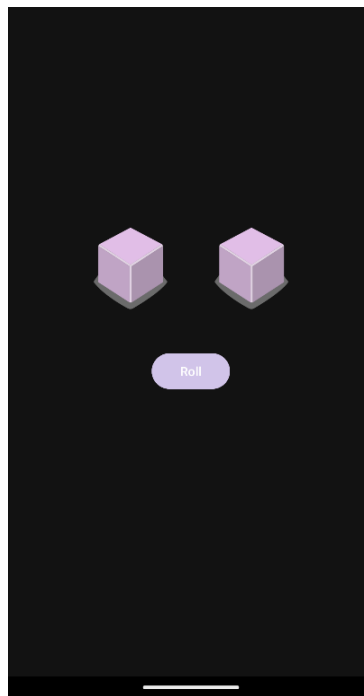
53     ) {
54
55         Row(
56             modifier = Modifier.padding(bottom = 32.dp),
57             horizontalArrangement = Arrangement.Center
58         ) {
59             Image(
60                 painter = painterResource(id =
getDiceImage(diceValue1)),
61                 contentDescription = "Dice 1",
62                 modifier = Modifier
63                     .size(120.dp)
64                     .padding(end = 16.dp)
65             )
66             Image(
67                 painter = painterResource(id =
getDiceImage(diceValue2)),
68                 contentDescription = "Dice 2",
69                 modifier = Modifier.size(120.dp)
70             )
71         }
72
73         Button(
74             onClick = {
75                 diceValue1 = (1..6).random()
76                 diceValue2 = (1..6).random()
77
78                 if (diceValue1 == diceValue2) {
79                     Toast.makeText(context, "Selamat,
anda dapat dadu double!", Toast.LENGTH_SHORT).show()
80                 } else {
81                     Toast.makeText(context, "Anda belum
beruntung!", Toast.LENGTH_SHORT).show()
82                 }
83             },
84             colors =
ButtonDefaults.buttonColors(containerColor =
Color(0xFFD1C4E9))
85         ) {
86             Text(text = "Roll", color = Color.Black,
fontSize = 16.sp)
87         }
88     }
89 }
90
91 fun getDiceImage(value: Int): Int {
92     return when (value) {

```

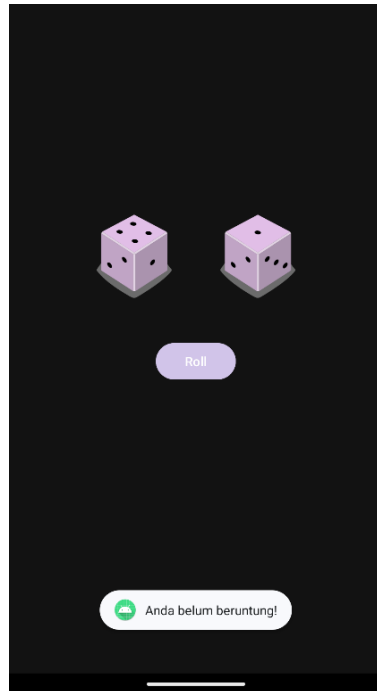
```
93         1 -> R.drawable.dice_1
94         2 -> R.drawable.dice_2
95         3 -> R.drawable.dice_3
96         4 -> R.drawable.dice_4
97         5 -> R.drawable.dice_5
98         6 -> R.drawable.dice_6
99         else -> R.drawable.dice_0
100     }
101 }
```

B. Output Program

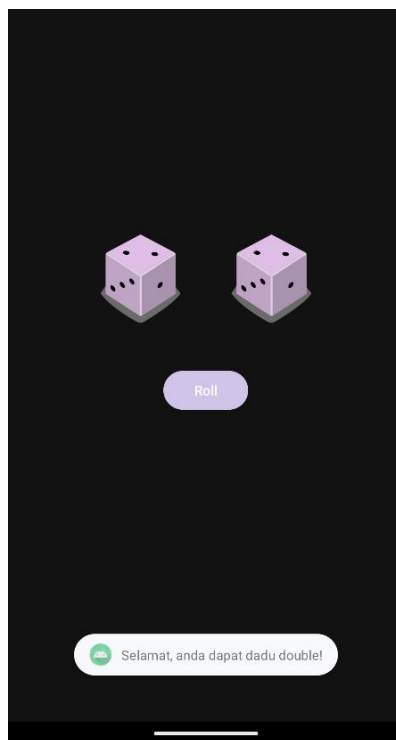
XML:



Gambar 4 Screenshot Hasil Jawaban Soal 1 XML

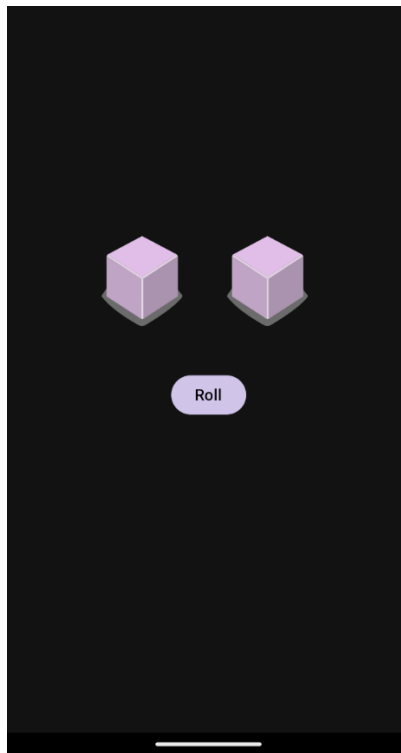


Gambar 5 Screenshot Hasil Jawaban Soal 1 XML

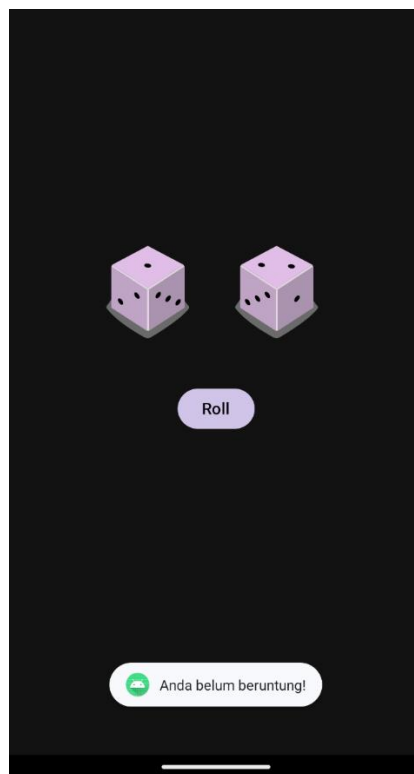


Gambar 6 Screenshot Hasil Jawaban Soal 1 XML

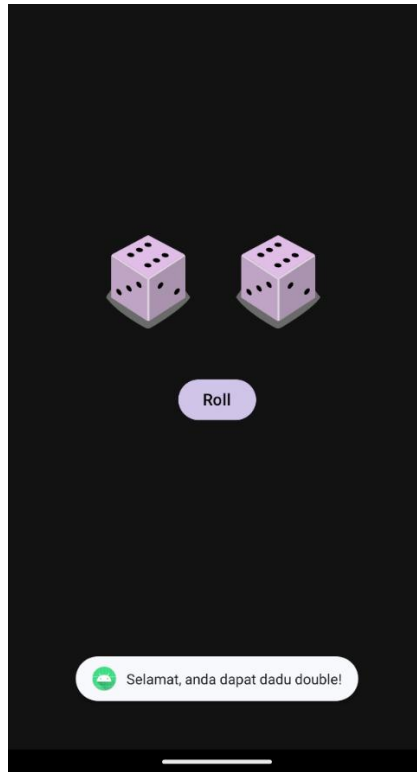
Compose:



Gambar 7 Screenshot Hasil Jawaban Soal 1 Compose



Gambar 8 Screenshot Hasil Jawaban Soal 1 Compose



Gambar 9 Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt XML:

Pada file `MainActivity.kt`, logika aplikasi mulai berjalan di dalam fungsi `onCreate()`. Fungsi `setContentView(R.layout.activity_main)` dipanggil untuk menghubungkan class Kotlin ini dengan desain XML yang telah dibuat. Lalu, dilakukan proses inisialisasi variabel menggunakan `findViewById` untuk menghubungkan komponen UI di XML dengan variabel di dalam kode Kotlin.

Aksi utama terjadi di dalam blok `btnRoll.setOnClickListener`. Ketika tombol "Roll" ditekan, program akan menghasilkan dua angka acak dari 1 hingga 6 menggunakan fungsi `(1..6).random()` dan menyimpannya di variabel `diceValue1` dan `diceValue2`. Angka acak tersebut kemudian dilempar ke fungsi pembantu bernama `getDiceImage()`. Fungsi `getDiceImage()` ini menggunakan struktur percabangan `when` untuk mengonversi nilai angka menjadi referensi gambar dadu yang sesuai. Setelah gambar didapatkan, fungsi `setImageResource()` digunakan untuk memperbarui gambar pada layar. Terakhir, program menggunakan percabangan `if-else` untuk membandingkan nilai kedua dadu; jika nilainya sama, aplikasi akan memunculkan pesan

"Selamat, anda dapat dadu double!" menggunakan Toast, dan jika berbeda, akan muncul pesan "Anda belum beruntung!".

activity_main.xml:

Pada file `activity_main.xml`, tata letak aplikasi dibangun menggunakan `LinearLayout` sebagai root layout. Root layout ini diatur dengan atribut `android:orientation="vertical"` agar komponen di dalamnya tersusun dari atas ke bawah, serta `android:gravity="center"` untuk memastikan semua elemen berada tepat di tengah layar. Warna latar belakang aplikasi juga diatur menjadi gelap menggunakan nilai hex color `#121212`.

Di dalam root layout tersebut, terdapat sebuah `LinearLayout` kedua yang memiliki orientasi horizontal yang berfungsi sebagai wadah untuk menyejajarkan dua buah gambar dadu secara berdampingan. Masing-masing dadu direpresentasikan oleh komponen `ImageView` yang telah diberikan ID unik (`@+id/ivDice1` dan `@+id/ivDice2`) agar dapat dipanggil pada sisi logika Kotlin nantinya. Secara bawaan, atribut `android:src` pada kedua `ImageView` diatur ke `@drawable/dice_0` sehingga tampilan awal saat aplikasi dijalankan adalah dua buah dadu kosong. Terakhir, di bawah wadah dadu tersebut, ditambahkan sebuah komponen `Button` dengan ID `@+id/btnRoll` yang berfungsi sebagai tombol pemicu untuk mengacak nilai dadu.

MainActivity.kt Compose

Kode berjalan mulai dari fungsi `onCreate()` yang memanggil `setContent {}`, berfungsi sebagai jembatan untuk merender komponen Compose bernama `DiceRollerApp()` ke layar.

Di dalam fungsi `DiceRollerApp()`, aplikasi menerapkan konsep State Management menggunakan variabel `diceValue1` dan `diceValue2` yang dibungkus dengan method `remember { mutableIntStateOf(0) }`. Penggunaan state ini sangat krusial dalam paradigma deklaratif, karena setiap kali nilai variabel ini, Compose akan secara otomatis melakukan recomposition untuk memperbarui tampilan gambar dadu tanpa perlu diperintahkan secara manual. Di sini juga dipanggil `LocalContext.current` untuk menyimpan context yang dibutuhkan saat menampilkan pesan Toast.

Untuk menyusun tampilannya, program menggunakan fungsi `Column` yang bertindak seperti `LinearLayout vertical` untuk menyusun elemen ke bawah, dilengkapi `Modifier` untuk membuat ukurannya memenuhi layar `fillMaxSize()`, memberi warna latar belakang gelap `background(Color(0xFF121212))`, dan memosisikan isinya ke tengah. Di dalam `Column`, terdapat fungsi `Row` yang membungkus dua komponen `Image`. Komponen `Image` tersebut memanggil fungsi `painterResource(id = getDiceImage(diceValue))` untuk menampilkan gambar yang sesuai dengan nilai state dadu saat itu.

Di bawah `Row`, terdapat sebuah fungsi `Button`. Pada properti `onClick` milik tombol ini, diletakkan logika untuk mengacak nilai dari 1 sampai 6 untuk kedua state dadu, serta percabangan `if-else` untuk mengecek apakah nilai dadu tersebut ganda atau tidak untuk menampilkan pesan `Toast` yang sesuai. Karena UI langsung terikat dengan nilai state, layar akan langsung memperbarui gambar dadu sesaat setelah nilai variabel di-set pada blok `onClick` tersebut.

D. Tautan Git

<https://github.com/rikafaulianarahmi/MODUL-MOBILE/tree/main>

ESSAY:

1. Dari hasil riset saya, Imperative Programming itu ibarat memberi instruksi langkah demi langkah ke komputer. Dalam pembuatan UI misalnya pakai XML dan Kotlin/Java, kita harus mencari ID komponennya secara manual, lalu mengubah keadaannya satu per satu seperti memanggil `setText()` atau `setImageResource`. Intinya, kita fokus pada bagaimana cara UI tersebut harus berubah.

Sebaliknya, Declarative Programming jauh lebih simpel. Hanya perlu mendeklarasikan apa yang ingin kita tampilkan berdasarkan data atau state saat ini. Contohnya pada Jetpack Compose, disitu tidak perlu repot menyuruh UI untuk berubah. Ketika datanya berubah, sistem akan secara otomatis menggambar ulang atau `recompose` bagian UI yang terkait.

Opini Paradigma:

Berdasarkan riset dan tugas praktikum yang sudah dilakukan, saya lebih prefer menggunakan Declarative Programming. Alasannya, penulisan kodenya jauh lebih ringkas dan saya tidak perlu lagi melakukan sinkronisasi antara data dan UI secara manual. Di pendekatan imperatif, sering kali terjadi bug karena kemungkinan mengubah data tapi lupa memperbarui UI-nya. Dengan paradigma deklaratif, masalah tersebut bisa dihindari karena UI selalu otomatis sinkron dengan data.

Scott, M. L. (2015). *Programming Language Pragmatics* (4th ed.). Morgan Kaufmann.

Hanus, M., & Kluß, C. (2009). Declarative Programming of User Interfaces. *Proceedings of the 18th International Conference on*

2. Kotlin dan Java sama-sama berjalan di atas JVM (Java Virtual Machine). Tapi ada perbedaan OOP antara keduanya yaitu:

- Null Safety, Kotlin langsung memisahkan variabel yang boleh bernilai kosong (null) dan yang tidak boleh. Jadi, kita bisa terhindar dari error NullPointerException (NPE) yang sering menjadi momok di Java.
- Tidak ada tipe primitif murni, di Java punya tipe data seperti int dan double, di Kotlin semuanya diperlakukan sebagai objek sejak awal.
- Setter Getter, di Java biasanya membuat fungsi get dan set panjang-panjang untuk membungkus variabel. Di Kotlin, ini sudah otomatis diurus menggunakan fitur Properties.
- Primary Constructor yang ringkas, di Kotlin bisa langsung menulis parameter class di sebelah nama classnya, jadi kodenya tidak bertele-tele dan jauh lebih rapi dibanding deklarasi constructor di Java.

Jemerov, D., & Isakova, S. (2017). *Kotlin in Action*. Manning Publications.

3. SOLID adalah singkatan dari lima prinsip dasar untuk membuat code yang rapi, mudah dibaca, dan gampang dikembangkan di masa depan.

- S - Single Responsibility Principle (SRP), satu class hanya punya satu tugas saja. Misalnya, class untuk mengurus tampilan UI jangan dicampur dengan logika untuk mengambil data dari database.

- O - Open/Closed Principle (OCP), code seharusnya gampang kalau mau ditambah fitur baru (Open), tapi jangan sampai harus mengubah code inti yang sudah berjalan dengan baik (Closed).
- L - Liskov Substitution Principle (LSP), class anak (subclass) harus bisa menggantikan class induknya (superclass) tanpa membuat program menjadi error atau berjalan tidak semestinya.
- I - Interface Segregation Principle (ISP), lebih baik membuat banyak interface yang kecil dan spesifik untuk satu tujuan, daripada membuat satu interface raksasa yang isinya campuran fungsi yang tidak selalu dipakai.
- D - Dependency Inversion Principle (DIP), class yang tingkatannya tinggi tidak boleh bergantung secara langsung pada class yang tingkatannya rendah. Keduanya harus bergantung pada sebuah abstraksi seperti interface.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

4. Class biasa di Kotlin fungsinya seperti class OOP pada umumnya. Kalau ingin membuat class yang tugasnya hanya untuk menyimpan data seperti data biodata atau user, programmer harus menulis sendiri fungsi-fungsi tambahannya secara manual.

Sedangkan Data Class hanya dengan menambahkan kata data di depan deklarasi class, Kotlin akan otomatis membuatkan fungsi-fungsi pendukung di belakang layar. Otomatis langsung punya fungsi `toString()` untuk mencetak data dalam bentuk teks yang mudah dibaca, `equals()` dan `hashCode()` untuk membandingkan isi datanya, `copy()` untuk menyalin objek dengan mudah, dan `componentN()` untuk memecah objek menjadi variabel terpisah secara langsung. Jadi, data class sangat menghemat baris kode atau boilerplate ketika programmer hanya butuh wadah untuk menyimpan data.